Collaboration Is Composition

Mealy Machines over Signed Records in Poly and Org

Risav Karna
CoSys Work Pvt. Ltd., cosys.work

November 2026

Abstract

People, organisations and the software acting for them increasingly work together without a shared server and without an orchestrator: each party keeps its own record, and work moves between peers as signed messages. We argue that such collaboration is composition. Every party acts through a Mealy machine over an append-only log of signed records, and a collaboration is existing machines wired together through mailboxes, so a collaboration is itself a machine and can be wired into a larger one. We give a mechanized model of one such system, the model that our published libraries implement, with primitives from a research matrix we wrote in 2014 and from applied category theory. In the category of polynomial functors a mailbox turns each machine into a dynamical system, and wiring that changes as units pair becomes one map whose routing depends on the units' positions. Held in a registry, the wiring carries the identitor and compositor of a dynamic organization in the sense of Shapiro and Spivak. Claims are types whose only inhabitants are records signed by the right keys, so an approval signed by an agent does not type-check. What composition gives follows from known results, applied to this model and checked in Lean 4. Bisimilar machines give bisimilar collaborations. When every machine is monotone in its log, logs and views converge under any fair delivery order, a form of the CALM theorem, and a step that is not monotone is where the model asks for a signature. Acceptance of a derived fact is a homomorphism out of a provenance semiring, and trust is a fibration over observers, with each ranking computed inside its observer's fiber. A case study from health care, in which a patient routes lab results and payment requests between her clinic and her family, runs through the paper.

Introduction

A family in Kathmandu pays for a mother's diabetes care while her son works abroad. Her lab sends a result, her clinic reads it, and her son is asked to pay a bill. Each of these parties keeps its own records, on its own devices, and none of them should have to trust a server in the middle to know who said what.

Our thesis is that collaboration among such parties is composition. Every party, which we call a *unit*, acts through one box: a Mealy machine whose inputs and outputs are signed records and whose state is a fold over an append-only log. A collaboration is a wiring of boxes that already exist, with a mailbox on each wire, and the result is again a box, so collaborations nest. Nothing outside the boxes and the wires coordinates them: the model has no central coordinator and no scores. The rest of the paper says what this view gives, and checks it. The next section traces how we arrived at it.

The model's primitives come from a research matrix we wrote in 2014 and from applied category theory: Mealy machines as coalgebras, wiring diagrams, polynomial functors and

Shapiro and Spivak's $\mathbb{O}\mathbf{rg}$, provenance semirings and fibrations. The mathematics is known. Each result applies an existing one: bisimulation [30] for machines composed in series, in parallel and with delayed feedback [21, 22], the CALM theorem [2, 14], the universal property of provenance semirings [12] and the Grothendieck construction [17]. What we add is a model of a system as it is built, stated precisely enough that these results hold of it, and a check of them in Lean 4. Stating the model that precisely changed it: one version let an agent key vouch for itself, and in another no new pairing could form inside a run. The definitions here are the corrected ones.

Contributions. The contributions are a model and its mechanization. Where an item rests on a known result, it names it.

1. A model of parties as append-only Mealy machines over signed records, and of collaboration as wiring through mailboxes, drawn as wiring diagrams with a delay on every wire. Records also arrive from outside: what people enter, and the records of pairing ceremonies (*The model*).

2. Wiring that changes as units pair and revoke, with Erlang's actor model as the reference, implemented in the category of polynomial functors. A mailbox makes each box a dynamical system, routing that follows the pairings is one map of polynomials, routing maps compose as partitions refine, and a registry of pairings gives the identitor and compositor of a dynamic multicategory in Shapiro and Spivak's $\mathbb{O}\mathbf{rg}$ (*The changing wiring in Poly*).

3. Claims as types: a claim kind is a dependent type whose inhabitants are records carrying the signatures its authority rule asks for. Reserved claims are uninhabited by agent keys, and an agent key counts only when a person who holds a unit has signed for it (*Claims as types*).

4. Congruence: bisimilarity of boxes is preserved by serial, parallel and mailbox-guarded feedback wiring under every delivery schedule, for the composition of machines of [21, 22] restated for records and schedules (Theorem 24).

5. Order independence, a form of the CALM theorem [2, 14]: when every box is monotone, the logs and views of a collaboration do not depend on delivery order or duplication (Theorem 26). With finitely many units and finitely many records that are input or emitted, the collaboration falls quiet at the least fixed point. With unboundedly many units, every unit's log settles there, though the network as a whole may never fall quiet (Proposition 27). Wiring that grows through pairing ceremonies keeps the result (Proposition 29). A step that is not monotone can depend on the order (Proposition 28), and the model asks for a co-signature there.

6. Grades without numbers: records are graded by the keys that signed them, derived facts carry provenance in the semiring of positive Boolean expressions [12], and each observer's acceptance is the unique homomorphism from it (Theorem 31).

7. Trust as a fibration over observers, a Grothendieck construction [17], and record sets that glue for federation (*Trust as a fibration over observers*).

8. A mechanization in Lean 4 of the propositions, lemmas and theorems of *The changing wiring in Poly* to *Trust as a fibration over observers*, published with the libraries that implement the model (*Artifact*).

How we arrived at the model

The model comes from three places: a research matrix from 2014, a notebook that recast four of its rows as the columns of a schema for a network, and the constraints we met building software for health care and for public posts.

The matrix

The matrix [19], written in August 2014, reads twelve concepts across language, computation and minds. Several of its rows anticipate this paper: nodes as small programs that form automata (row 1, the boxes), hypergraph meshes (row 2, the records), syncs and sinks (rows 6 and 7, convergence and the append-only log), collaborations (row 10, the wiring) and the equivalence of theory and practice (row 12, bisimulation). The model takes these rows as its primitives, and this paper states them precisely enough to check.

From rows to columns

A later notebook turned four of the rows into the columns of a schema for a network: Identity, Actors, Actions and Collaborations. Reading each column in category theory gave the structures of the model below. Identity became a base of key-holding units, with persons as roles inside them. An actor became a Mealy machine, a coalgebra over that base. An action became a record, a hyperedge whose incidences name what it is about. A collaboration became a tensor of existing actors in the operad of wiring diagrams, so working together only ever adds wires. Two further readings came out of practice: an assertion is graded by the key that signs it, and trust belongs to whoever holds it.

Constraints that chose the structures

Each structure answers a constraint we met while building (Table 1). Health care gave the first three: a patient's records stay with her, her lab results reach a clinic only through her, and family abroad pay a bill only on a request she starts and her clinic co-signs. An earlier product of ours for organisations gave the rule that assistants draft and only people sign. Public posts gave two more: no published number about a person, and trust that differs by reader. Running the same behaviour on more than one runtime, and bridging to other protocols, gave the last two.

Background

Mealy machines. A Mealy machine [25] is a tuple $(S, I, O, \delta, \lambda, s_0)$ with a transition function $\delta\colon S \times I \to S$ and an output function $\lambda\colon S \times I \to O$. A Moore machine [26] has $\lambda\colon S \to O$ instead, and so answers one step later. Currying gives the coalgebra $S \to (O \times S)^I$; the final coalgebra is the set of causal stream functions, and two states are behaviourally equivalent exactly when they are bisimilar [6, 30, 31].

Wiring. Machines compose in series, in parallel and with feedback [21, 22]. Spivak made the patterns themselves an operad, the operad of wiring diagrams [36]. Open dynamical systems in continuous time, with wires that pass their contents on at once, form an algebra on an operad of wiring diagrams [40], and Rupel and Spivak give a discrete-time version whose wires carry state [29]. Polynomial functors give the current account, with interfaces as

Constraint	Structure it led to	Where
Each party keeps its own record and works offline; no server is the source of truth	Append-only logs merged by union	Definition 3, Theorem 26
A lab result reaches a clinic only through the patient	Collaboration as wiring, with no lab-to-clinic wire	Definition 5, Figure 1
Family pay only on a request the patient starts and her clinic co-signs	Co-signatures as records; dependent claim types	Example 19
Assistants draft; only people sign	Reserved claim types and operated_by	Proposition 21
No published number about a person	Grades as sets of signers; provenance in PosBool	Theorem 31
Trust differs by reader	A fibration over observers	*Trust as a fibration over observers*
The same behaviour from more than one implementation	Bisimulation and congruence	Theorem 24
A new protocol must leave the graph as it was	Composition is wiring; a bridge is a box	Definition 5

Table 1: Constraints we met while building, and the structures they chose.

polynomials and wiring as lenses [28]. Wiring that changes with the state of its parts is treated in [38].

Actors. An actor [1, 15] handles one message at a time, may send messages, create actors and set its next behaviour. With its behaviour counted as state, an actor is a Mealy machine; Erlang's `gen_statem` documents itself this way [11]. Actors add creation, addresses passed as data, and delivery order chosen by the network.

Propositions as types. A proposition corresponds to a type and a proof to a program of that type [16, 41]. Dependent types let a proposition mention the values it is about.

Replicas and monotonicity. State that only grows by union is a state-based replicated data type [35], and programs that are monotone in their inputs are consistent without coordination [14]. Provenance semirings record how a derived fact depends on its sources [12].

The model

Records, units and roles

We use the record of [9]: a record is $r = (a, t, \kappa, I, c)$, with an author key a, a time t, a kind κ from a registered vocabulary, a finite list of role-labelled incidences $I = ((\rho_1, x_1), \ldots, (\rho_n, x_n))$ and a content c. Its identifier is $\mathsf{H}(a, t, \kappa, I, c)$ for a collision-resistant hash H, and the author signs the identifier. Records and their incidences form a hypergraph, read through its Levi graph, and the graph of references by identifier is acyclic. Write $\mathcal{R}$ for the set of records and K for the set of keys.

Definition 1 (Unit, role)**.** *A* unit *is a key-holding party: a clinic, a family, a team, an office. It holds its keys, a signed directory record saying who it is and where its mail goes, and its own replica of the records that concern it. A* role *is a position a person or an agent holds inside a unit. Persons appear in records only as roles.*

Definition 2 (Co-signature)**.** *A* co-signature *of a record r is a record s of kind* cosign *with one incidence $(\textsf{subject}, r)$ and one $(\rho, \cdot)$ naming the role in which its author signs. Records are never changed; a further signature on r is a further record.*

Boxes

Definition 3 (Box)**.** *A* box *for a unit u is a Mealy machine $B = (S, \mathcal{R}, \mathcal{R}^*, \delta, \lambda, s_0)$ whose inputs are records and whose outputs are finite sequences of records, each signed with a key of u or forwarded from its log. Forwarding passes on a record unchanged and signs nothing new. The box is* append-only *if it has a* log *map $\ell \colon S \to \mathcal{P}_{\text{fin}}(\mathcal{R})$ with*

$$\ell\big(\delta(s,r)\big) \;=\; \ell(s) \cup \{r\} \cup \operatorname{set}\big(\lambda(s,r)\big).$$

It is log-determined *if every reachable state is a function of its log: $s = \Phi(\ell(s))$ for some Φ.*

For an input sequence $r_1 \dots r_n$ the state is the fold $s_n = \delta(\dots\delta(\delta(s_0, r_1), r_2)\dots, r_n)$. A log-determined box's state is a function of its log. The log itself can still depend on the order of the inputs, since what the box emits may; for the monotone boxes of *Composition and order* it does not. Corrections and retractions are new records that point at old ones, so no record is edited and none is deleted; a view may still leave a retracted record out.

Mailboxes and collaborations

Definition 4 (Mailbox)**.** *A* mailbox *from unit u to unit v is a Moore machine whose state is a finite set of deposited records. Depositing adds a record; collecting hands one deposited record to v's box. Its output depends on its state alone.*

A mailbox is the delay that makes feedback safe. A reply from v to u goes through a second mailbox, so no output of a composite depends on itself within one step.

Definition 5 (Collaboration)**.** *Let W be a set of units, each with a box, and $P \subseteq W \times W$ a set of pairings between distinct units, each established by a co-signed pairing record. A unit needs no pairing with itself, since its log already holds what its box emits. The* collaboration *$C(W, P)$ is the wiring in which each record emitted by u's box whose audience includes v, with $(u, v) \in P$, is deposited in the mailbox from u to v. Each unit also has a mailbox from outside, holding its* outside inputs*: the records its people enter and the records of pairing ceremonies. A* schedule *σ chooses, at each step, which mailbox delivers next and which of its records. $C_\sigma(W, P)$ is the resulting deterministic machine.*

Remark 6. The wiring of $C(W, P)$ is a wiring diagram in the sense of [36], with each mailbox a delay box on its wire. Discrete steps and wires that carry state are the setting of the temporal wiring diagrams of [29]. A schedule is not part of the diagram, so we do not claim that collaborations form an algebra on either operad.

Wiring that changes

Erlang is our reference for a network whose shape changes as it runs [4, 39]. An Erlang process sends to the identifiers it holds, learns new ones from messages, spawns processes and links to them, and messages from one process to another keep their order. We keep spawning and learning, with one restriction: a unit sends only along a standing pairing, and a pairing stands only when both of its ends have signed it. Nothing in the model relies on order within a mailbox.

Definition 7 (Pairings of a log)**.** *For a log L, $P(L)$ is the set of pairs (u, v) of distinct units such that L holds a pairing record signed by u and co-signed by v, with no revocation of it in L. An* introduction *by a third unit w is a record proposing a pairing of u and v; it adds nothing to $P(L)$ until u and v have both signed, in a ceremony.*

Definition 8 (Collaboration with changing wiring)**.** *In $C(W)$ a record that u's box emits for v is deposited in the mailbox from u to v once $(u, v) \in P(\ell_u)$ for u's own log ℓ_u, and u holds it until then. A pairing is made in a* ceremony*: its two ends sign the pairing record and the co-signature together, in person or over a channel outside the model, and both records reach each end as outside inputs. Creating a unit, as an Erlang process spawns another, adds it to W with an empty log.*

Between two pairing events $C(W)$ is a collaboration $C(W, P)$ with fixed wiring, so the wiring at any moment is a fold over the pairing records in the logs. This is wiring that depends on the state of its parts, in the sense of [38], with the topology changed only by signed records; *The changing wiring in Poly* states it precisely.

The changing wiring in Poly

This section states the changing wiring in the category **Poly** of polynomial functors, where Spivak gives mode-dependent dynamics a categorical home [28, 37], and in Shapiro and Spivak's monoidal double category $\mathbb{O}$**rg** of dynamic organizations [34].

A polynomial $p = \sum_{I \in p(1)} y^{p[I]}$ has a set $p(1)$ of positions, its possible outputs, and at each position I a set $p[I]$ of directions, the inputs it accepts there. A map $\phi\colon p \to q$ sends each position I of p to a position $\phi(I)$ of q, and each direction $j \in q[\phi(I)]$ back to a direction $\phi(I, j) \in p[I]$ [34]. The backward part depends on the position, which is how wiring can depend on the state of the parts. A dynamical system with interface p and states S is a map $Sy^S \to p$ [28], and the tensor $p \otimes q = \sum_{(I,J)} y^{p[I] \times q[J]}$ runs systems side by side.

Rounds

To keep every system deterministic we run collaborations in rounds. In each round every unit receives, as one finite set, the outside inputs and the records deposited for it along standing pairings since the last round, and its box takes that set in one step. We write $\delta(s, X)$ for the box's next state and $\lambda(s, X)$ for what it emits on a set X; a box that reads one record at a time folds δ over X.

Proposition 9 (Rounds are a fair schedule)**.** *For monotone boxes, running in rounds reaches the same logs as every fair schedule of Theorem 26.*

Proof sketch. A monotone box reaches the same log whatever order it folds a set in, a record deposited in one round is delivered in the next, and duplicate deliveries change nothing. So rounds are a fair schedule. □

Units as dynamical systems

An *envelope* is a record together with one unit of its audience, its recipient. A box emits a record as one envelope for each unit of its audience other than its own, and handing a box an envelope hands it the record. Write $\mathcal{E}$ for the envelopes and $\mathcal{E}_W$ for those addressed to members of W.

Definition 10 (Interfaces)**.** *For a finite set W of units, let Pos_W be the set of pairs (P, D) in which P is a finite set of pairings in which a member of W shares with a unit outside W, and D a finite set of envelopes that members of W hold for units outside W. Then*

$$p_W \;=\; \sum_{(P,D)\in \mathrm{Pos}_W} y^{\mathcal{P}_{\mathrm{fin}}(\mathcal{E}_W)}.$$

A unit u has the interface $p_u = p_{\{u\}}$: its position is its standing pairings and its deposits, and in each round it accepts a finite set of envelopes addressed to it.

Proposition 11 (A unit with its mailboxes is a dynamical system)**.** *Let B be a box for u with states S, run in rounds. Together with its outgoing mailboxes it is a map $\varphi_u \colon S_u y^{S_u} \to p_u$ with $S_u = S \times \mathcal{P}_{\mathrm{fin}}(\mathcal{E})$, a state of the box together with the envelopes it has emitted and not yet delivered. At (s, E) the position holds the pairings in $\ell(s)$ in which u shares with another unit and the envelopes in E that u holds for other units. The next state on a set X is $(\delta(s, X),\ (E \setminus E_s) \cup \lambda(s, X))$, where E_s holds the envelopes of E whose pairing stands at s, which routing delivers in this round. Started with nothing deposited, after n rounds the first component is the state of B after those rounds, and the deposits are the envelopes B emitted in an earlier round whose pairing has not stood in any round since.*

Proof sketch. The outgoing mailboxes are Moore machines, so what the unit shows the network, its pairings and its deposits, is a function of its state alone. That is the shape of a map $S_u y^{S_u} \to p_u$: the position is read off the state, and the update takes the state and a direction. The statement about n rounds follows by induction on n, since each round drops the envelopes it delivers and adds what the box emits in it. So each envelope is delivered once. □

A Mealy box on its own has no map of this shape, since its output depends on the input of the same step. The mailbox supplies the delay, as it did for feedback.

Collaborations as maps of polynomials

Definition 12 (Routing)**.** *Let $W = W_1 \sqcup \cdots \sqcup W_k$. The routing map $\omega \colon p_{W_1} \otimes \cdots \otimes p_{W_k} \to p_W$ sends positions $((P_1, D_1), \ldots, (P_k, D_k))$ to the pairings in $\bigcup_i P_i$ that leave W and the envelopes in $\bigcup_i D_i$ addressed outside W. Given those positions and a set $X \subseteq \mathcal{E}_W$ arriving from outside, it gives the group W_i the direction*

$$X_i \;=\; (X \cap \mathcal{E}_{W_i}) \;\cup\; \{\, e \in D_j \cap \mathcal{E}_{W_i} : j \neq i,\ (\sigma(e), \tau(e)) \in P_j \,\},$$

where $\sigma(e)$ is the sender of e, $\tau(e)$ its recipient, and $(a, b) \in P_j$ says that the pairing in which a shares with b stands.

The backward part reads the pairings P_j out of the positions, so ω is one map of polynomials whose routing changes with the state of the groups. Spivak and Tan's modes [38] appear here as positions.

Theorem 13 (Changing wiring as a map of polynomials)**.** *(i) Let $u_1, \ldots, u_n$ be distinct units, $W = \{u_1, \ldots, u_n\}$, and ω the routing map for the partition of W into single units. With nothing arriving from outside, one step of the composite system $\omega \circ (\varphi_{u_1} \otimes \cdots \otimes \varphi_{u_n})$, from any state, is one round of the collaboration with changing wiring of Definition 8: each unit receives every envelope deposited for it by a unit whose log holds the pairing in which that unit shares with it, and the sender stops holding it. (ii) The routing map of the partition of W into one group is the identity on p_W, and routing nests: if each W_i is in turn split as $W_{i1} \sqcup \cdots \sqcup W_{im_i}$, the routing map for $\{W_i\}$ composed with the tensor of the routing maps for each $\{W_{ij}\}_j$ is the routing map for $\{W_{ij}\}_{i,j}$. Both hold up to the canonical isomorphisms $p \cong \bigotimes_{\{*\}} p$ and $\bigotimes_i \bigotimes_j p_{ij} \cong \bigotimes_{(i,j)} p_{ij}$. So routing maps compose as partitions refine, with the interfaces p_W as objects, and a collaboration is itself a unit of a larger one.*

Proof sketch. (i) In one round, both deliver to each unit the envelopes deposited for it whose sender's log holds their pairing. Since the units are distinct, the pairing ω reads from the positions is the one in the sender's own log. An envelope that Definition 8 holds back is one that ω does not yet route, and the sender drops exactly the envelopes ω routes. (ii) For one group nothing is routed inside and everything crosses the boundary, so both parts are identities. For nesting, an envelope between two parts of one W_i is routed by the inner map and one between different W_i by the outer map. Either way it reaches its recipient in the same round, and both sides expose the same pairings and deposits at the boundary of W. The case split uses that the groups are disjoint: an envelope deposited by one part is sent along a pairing that part holds. □

The wiring as a dynamic organization

In ω the pairings are read from the units' positions, so the units are the only holders of state, which is our design. Shapiro and Spivak's $\mathbb{O}\mathbf{rg}$ lets the wiring be a system of its own. Its objects are polynomials, and a horizontal morphism from p to q is a $[p, q]$-coalgebra, a set S with a map $\beta\colon S \to [p, q](S)$, where

$$[p, q] \;=\; \sum_{\phi\colon\, p \to q} y^{\sum_{I \in p(1)} q[\phi(I)]}$$

is the internal hom for $\otimes$ [34]. At each state such a coalgebra puts out a map $\phi\colon p \to q$ and takes in what flows through it: a position I of p and a direction of q at $\phi(I)$.

Definition 14 (Pairing registry)**.** *For $W = W_1 \sqcup \cdots \sqcup W_k$, the registry is the $[p_{W_1} \otimes \cdots \otimes p_{W_k}, p_W]$-coalgebra whose states are sets Q of pairings between members of different groups. At state Q it puts out the routing map ω^Q, which routes between groups along the pairings in Q, as in Definition 12 with Q in place of the pairings read from the positions. The pairings to units outside W are still read from the positions. On a flow (I, X) it moves to the set of pairings between groups that the positions I show.*

Proposition 15 (Registries as a dynamic organization)**.** *The registries carry the identitor and compositor of a dynamic multicategory in the sense of Shapiro and Spivak [34, Definition 3.2], with objects the interfaces p_W. The identitor sends the one state of the identity coalgebra on p_W to the empty state of the registry of W as one group. The compositor sends a state of the nested registries, the registry for $\{W_i\}$ composed with the tensor of the registries for each $\{W_{ij}\}_j$, to the union of their states in the registry for $\{W_{ij}\}_{i,j}$. Both are maps of coalgebras, the compositor read along the isomorphism that flattens the tensor, so each is a square in $\mathbb{O}\mathbf{rg}$. On the sets of pairings the states hold, the compositor is unital and associative.*

Proof sketch. At the empty state the registry of one group routes nothing inside and passes everything across, so it puts out the identity. With one group there are no pairings between groups, so it stays empty. A pairing between two parts of the flattened partition joins either two groups W_i, and the outer registry holds it, or two parts of one W_i, and that group's registry holds it. So the union of the nested states is a state of the flattened registry, and its routing map is the nested one, as in Theorem 13(ii). After a flow, the union of the nested updates is the update of the union, since both hold the pairings between parts that the positions show. Union is associative and has the empty set as unit, which gives the equations of [34, Definition A.2] on the pairings the states hold. □

Reading the pairings from positions, as ω does, and holding them in a registry are two presentations of the wiring. The first keeps every piece of state in a unit's log; the second is the form in which $\mathbb{O}\mathbf{rg}$ composes it. They are not interchangeable. The registry routes along

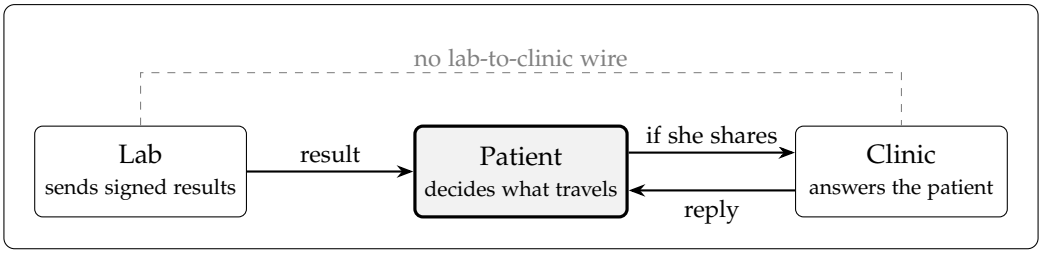


Figure 1: The lab-result collaboration. Each arrow is a mailbox carried by a courier that cannot read it. Every connection runs through the patient.

the pairings the positions showed one round earlier, so after a revocation it can still deliver along a pairing that no longer stands, and a box that records when a record arrived can tell the two apart for good.

Claims as types

Definition 16 (Accountable key)**.** *The directory partitions keys into those held by persons and those held by agents, $K = K_{\text{person}} \uplus K_{\text{agent}}$, and says which person keys hold which units. A key k is* accountable *in a log L when a person holds it, or when L holds an* operated_by *record for k that names a unit and is signed by a person key that holds that unit. We write Acc_L for the keys accountable in L. An agent key cannot vouch for itself.*

Definition 17 (Grade of a record)**.** *For a log L, the* grade *of $r \in L$, with author key a, is its author, in the role ρ_r that r names for its author, together with every co-signer whose key is accountable in L:*

$$g_L(r) = \{(\rho_r, a)\} \cup \{(\rho, k) : \exists s \in L,\ s \text{ co-signs } r \text{ in role } \rho \text{ with key } k \in \mathsf{Acc}_L\},$$

a subset of Roles $\times K$.

Definition 18 (Claim type)**.** *A* claim kind κ *has parameters $x_1{:}X_1, \ldots, x_n{:}X_n$ and an* authority rule $\mathsf{Auth}_\kappa(\bar{x}, L) \subseteq \text{Roles} \times K$*, the signatures the claim needs, which may depend on the parameters and on pairing records in L. The* claim type *is*

$$\mathsf{Claim}_\kappa(\bar{x})_L = \{\, r \in L : a(r) \in \mathsf{Acc}_L,\ \kappa(r) = \kappa,\ r \text{ names } \bar{x} \text{ in } \kappa\text{'s roles},\ \mathsf{Auth}_\kappa(\bar{x}, L) \subseteq g_L(r) \,\},$$

where $a(r)$ is the author key of r. A record signed by an agent key for which no person has signed an operated_by *record in L inhabits no claim type, and a co-signature with such a key counts for nothing.*

Example 19. $\mathsf{Consented}(p, x, c)$, "patient p shares result x with clinic c", needs p's key in the role subject, which her consent record names for its author. $\mathsf{PayRequest}(p, c, \mathit{item})$ needs p's key as requester, the role her request names for its author, and, as cosigner, the key of the clinic c that p is paired with in L: the authority depends on a pairing record, so the type is dependent. A request the clinic has not co-signed is a record in L that does not inhabit the type.

Proposition 20. *(i) If L is finite, naming is decidable, and $\mathsf{Auth}_\kappa(\bar{x}, L)$ is a finite set computable from $\bar{x}$ and L, then inhabitation of $\mathsf{Claim}_\kappa(\bar{x})_L$ is decidable from L and the directory records. (ii) If $\mathsf{Auth}_\kappa(\bar{x}, L)$ asks for a key that signed no record of L, the claim type is empty. (iii) If Auth_κ never asks for more as the log grows, that is $L \subseteq L'$ implies $\mathsf{Auth}_\kappa(\bar{x}, L') \subseteq \mathsf{Auth}_\kappa(\bar{x}, L)$, then $L \subseteq L'$ implies $\mathsf{Claim}_\kappa(\bar{x})_L \subseteq \mathsf{Claim}_\kappa(\bar{x})_{L'}$.*

Proof sketch. (i) Every condition in Definition 18 is a check against the finitely many records of L and the finitely many signatures Auth_κ asks for. (ii) A grade holds only keys that signed records of L, as authors or as co-signers. For (iii), Acc_L and $g_L(r)$ grow with L, and the condition keeps Auth_κ from growing. A revocation can make Auth_κ ask for more, which is the kind of step Proposition 28 covers. □

Agents draft, people sign. A claim kind is *reserved* when its authority rule asks for at least one signature, and only for person keys.

Proposition 21. *(i) Every key that signs a record of a claim type, as its author or as a co-signer counted in its grade, is accountable: a person holds it, or it reaches a unit a person holds by one* operated_by *record that person signed. (ii) No log whose records are all signed by agent keys inhabits a reserved claim type.*

Proof sketch. (i) follows from Definition 18: the claim type asks for an accountable author, and the grade counts only accountable co-signers. (ii) A reserved rule asks for some person key, and in a log signed only by agent keys no grade contains one. □

An agent's outbox can carry drafts and proposals. An approval, a payment or a decision exists only when a person with the right key signs it.

Definition 22 (Placement of an agent)**.** *An agent working for one principal is a* role *inside that principal's unit and signs with its own agent key. An agent working for several principals is a* unit *of its own, with a mailbox and agent keys. In both placements the agent's key has an* operated_by *record naming the unit that answers for it, signed by the person who holds that unit.*

Lemma 23 (Placement changes routing only)**.** *Suppose every key is accountable in both placements alike, and no authority rule reads the records a placement writes: the unit, its mailbox, the role and the* operated_by *record. Then, when an agent moves from one placement to the other, every other record inhabits the same claim types.*

Proof sketch. Inhabitation depends on the other records of L, on Auth_κ and on Acc_L (Definition 18). The records a placement writes co-sign nothing, so every grade is the same in both logs, and the hypotheses fix the rest. □

What the move changes is the wiring: which unit's mailboxes the agent's records pass through.

Composition and order

Theorem 24 (Congruence)**.** *Let W be a set of units and P a set of pairings, and let each unit u have two boxes B_u and B'_u with B_u bisimilar to B'_u. Then for every schedule σ, $C_\sigma(W, P)$ built from the B_u is bisimilar to $C_\sigma(W, P)$ built from the B'_u.*

Proof sketch. The product of the component bisimulations, extended by equality on mailbox states, is a bisimulation of the composites: matching states receive equal records from equal mailbox states, emit equal records, and move to matching states. Feedback is well defined because every loop passes through a mailbox, whose output does not depend on its current input. The composition is that of Mealy machines in series, in parallel and with delayed feedback [21, 22], here for records and schedules. □

Definition 25 (Monotone box)**.** *An append-only, log-determined box is* monotone *if there is a monotone map* $\mathsf{Out}\colon \mathcal{P}_{\text{fin}}(\mathcal{R}) \to \mathcal{P}_{\text{fin}}(\mathcal{R})$ *with* $\mathsf{Out}(L \cup \mathsf{Out}(L)) = \mathsf{Out}(L)$ *for every* L*, such that* $\mathsf{Out}(\ell(s_0)) = \emptyset$ *and the box follows the* receive rule*: on an input* r *at a state with log* L *it emits the records of* $\mathsf{Out}(L \cup \{r\})$ *that it has not emitted before. On an infinite set* X *we read* $\mathsf{Out}(X)$ *as the union of* $\mathsf{Out}(L)$ *over the finite* $L \subseteq X$*.*

Under the receive rule the log after an input r is $L \cup \{r\} \cup \mathsf{Out}(L \cup \{r\})$, and the records emitted up to any state s are exactly $\mathsf{Out}(\ell(s))$. The condition on Out says that the box's own emissions never make it emit more. A monotone box reads a set of records to the same state in whatever order it reads them, and reading a record again changes nothing. Without the rule, a box whose emissions need only lie in Out of its log can emit a record that justifies itself, and then the order of its inputs decides its log.

Theorem 26 (Order independence)**.** *Let every box in* $C(W, P)$ *be monotone, and let schedules be fair: every record that is ever waiting in a mailbox, outside inputs included, is collected at some step while it waits. Schedules may deliver a record more than once. Let* μF *be the least fixed point above the initial logs and the outside inputs, defined in the proof.*

(i) Finitely many units. *If* W *is finite and only finitely many records are outside inputs or can be emitted, every fair schedule reaches* μF *and stays there. Any two fair schedules end with the same logs, and so with the same value of any view that is a function of a log.*

(ii) Unboundedly many units. *If only finitely many records are outside inputs or can be emitted and* W *is any set of units, then along every fair schedule each unit's log settles on its component of* μF*. Any two fair schedules agree unit by unit.*

(iii) Unboundedly many records. *Along every fair schedule the union of each unit's logs over all steps is its component of* μF*, and so is any view of that union.*

A record can be emitted when it lies in Out_u *of the component of* μF *at some unit* u*.*

Proof sketch. Order the tuple of logs, one per unit, by inclusion, and let F add to each log its initial records, its outside inputs, the records that can be delivered to it, and the records its Out then yields. F is monotone. Under the receive rule every stage of a schedule lies below μF, by induction on the steps, and a duplicate delivery changes nothing. (ii) μF adds to each initial log only outside inputs and records that can be emitted, so each unit's log grows inside its initial log and a finite set, and settles after finitely many steps. Once every log a record depends on has settled, fairness delivers the record, so the settled logs are closed under F and equal μF [2, 14]. (i) With finitely many units the logs settle together, at one step. (iii) Read through its finite subsets, Out preserves unions of increasing chains, and a fixed pairing is decided at once. The union of the logs along a fair schedule lies below μF, and every record that F adds to it is already in some stage, so the union is closed under F and equals μF [10]. □

With unboundedly many units the network as a whole may never fall quiet, even though every unit does.

Proposition 27 (No global quiescence)**.** *There is a collaboration with infinitely many units, monotone boxes, fixed wiring and a single record, with a fair schedule along which the logs at every step differ from* μF*.*

Proof sketch. Let one unit receive the record as an outside input and share it with every other unit. Let the schedule deliver that input first and then collect from each mailbox infinitely often, one delivery per step. After n steps at most n units hold the record. □

Proposition 28. *Let unit u's box follow the receive rule with a map* Out *that holds o on every log that holds a record t without a record $x \neq t$, and on no log that holds x, as with "no objection by the deadline" or "not revoked". Let pairings only grow, and let there be countably many units and records. If at some step of a run t and x are both waiting in u's mailboxes, u's log does not hold x and u has not emitted o, then the run can be continued by two fair schedules: under one u emits o, and under the other it never does.*

Proof sketch. Deliver t next in one schedule and x next in the other, and then offer every delivery infinitely often, which countability allows. Since pairings only grow, a waiting record keeps waiting, so both schedules are fair. Once x is in u's log it stays there, so u never emits o in the second schedule. □

Without the hypothesis that x arrives the conclusion fails: if x never reaches u, every fair schedule emits o, and a condition that is not monotone has one outcome.

Proposition 29 (Order independence with growing wiring)**.** *Let Z be a set of records that holds every initial log and outside input and is closed under every* Out*, such as the records that are not revocations when no box emits one. If P is monotone on logs inside Z, Theorem 26 holds for the collaboration $C(W)$ of Definition 8, with P read on Z. For part (iii), each pairing must also be decided by some finite stage of an increasing chain of logs, as it is when a pairing record decides it.*

Proof sketch. Every log and every emission of a run lies in Z, so reading P on Z changes no run. On logs inside Z each $P(\ell_u)$ only grows, so a record held for want of a pairing is deposited once the pairing arrives and is never withdrawn. Deposits then depend monotonically on the logs, F stays monotone, and the argument for Theorem 26 applies. □

The wiring grows inside a run: a pairing whose records arrive as outside inputs, from a ceremony, opens a wire that was not there at the start, and every fair schedule then delivers along it.

Revocation removes a wire, so it is one of the steps Proposition 28 covers. Such steps are where the model asks for a coordinating signature, such as a co-signed receipt or a witnessed timestamp, which fixes the order. Key epochs, as the SOMA white paper uses for private circles [9], are a candidate for revocation in later work. References by identifier handle the rest: a co-signature names the request it signs, so it takes effect only once the request is present, in whatever order the two arrive.

Grades and acceptance without numbers

A view derives facts from records. Each derived fact f carries its provenance $\mathsf{prov}(f)$, a positive Boolean expression over the records it was derived from: conjunction for premises used together, disjunction for alternative derivations. Write $\mathsf{PosBool}[L]$ for these expressions up to equivalence [12].

Definition 30 (Acceptance)**.** *An observer m has a valuation $\nu_m \colon L \to \mathbb{B}$: $\nu_m(r) = 1$ when the author of r is accountable in L, r names parameters $\bar{x}$ of its kind κ, and the signatures in $g_L(r)$ that m trusts for their roles cover $\mathsf{Auth}_\kappa(\bar{x}, L)$. A record m accepts inhabits $\mathsf{Claim}_\kappa(\bar{x})_L$. The acceptance of a derived fact is $\hat{\nu}_m(\mathsf{prov}(f))$.*

Theorem 31 (Grades)**.** *(i) $\mathcal{P}_{\mathrm{fin}}(\mathrm{Roles} \times K)$ ordered by inclusion is a generalized Boolean lattice: distributive, with a bottom and relative complements, and with no top, since keys are unbounded. For a finite log L, $g_L(r)$ is finite, and it is monotone in L. (ii)* $\mathsf{PosBool}[L]$ *is a bounded distributive lattice,*

and so a commutative semiring with $\vee$ and $\wedge$ in which both operations are idempotent. (iii) ν_m extends uniquely to a homomorphism $\hat{\nu}_m \colon \mathsf{PosBool}[L] \to \mathbb{B}$, so an observer's acceptance of every derived fact is computed from its acceptance of records, and two observers who accept the same records accept the same derived facts.

Proof sketch. (i) Finite sets are closed under union, intersection and difference, and an infinite set has no largest finite subset. A grade grows with L because Acc_L does. (ii) and (iii) follow from $\mathsf{PosBool}[L]$ being the free bounded distributive lattice on L and from the universal property of provenance semirings [12]. □

Remark 32. No construction here adds a count. Replacing PosBool by the semiring of polynomials with natural coefficients would give counts and with them a score, and the model leaves that out on purpose. Quantitative type theory grades the variables of a judgement by their usage, drawn from a semiring [5]. Grading by sets of signers in the same way is a route to a type system with these grades built in.

Trust as a fibration over observers

Trust is always someone's trust, so we index it by the observer.

Definition 33 (Observers)**.** **Obs** *is the free category on the graph whose objects are units and which has an arrow $f \colon v \to u$ for each standing pairing in which u shares with v.*

Definition 34 (Fibers and reindexing)**.** *For a unit u, $T(u)$ is the set of sets of records whose audience includes u, ordered by inclusion. Trust assertions are records, so u's view of trust, the assertions about keys in its log whose audience includes u, is one object of $T(u)$. For $f \colon v \to u$, reindexing $f^* \colon T(u) \to T(v)$ keeps the records whose audience includes v, and along a path it does so once for each unit the path passes through. Reindexing preserves identities and composites on the nose, so it is a functor $T \colon \mathbf{Obs}^{\mathrm{op}} \to \mathbf{Cat}$.*

Proposition 35. *The Grothendieck construction $\int T \to \mathbf{Obs}$ is a split fibration: for $f \colon v \to u$ and X in $T(u)$, the morphism $(v, f^*X) \to (u, X)$ over f is cartesian, and these chosen lifts compose strictly.*

Proof sketch. The objects of $\int T$ are pairs (u, X) with X in $T(u)$, and a morphism $(v, Y) \to (u, X)$ is a path $f \colon v \to u$ with $Y \subseteq f^*X$. Composition is composition of paths, which is well defined because reindexing is monotone and functorial. A morphism $(w, Z) \to (u, X)$ over $f \circ g$ gives $Z \subseteq (f \circ g)^*X = g^*f^*X$, which is a morphism into the lift over g, and it is unique because each fiber is a poset [17]. □

Rankings. An observer m computes any ranking from its own fiber $T(m)$, and the model never publishes one: no record carries a ranking, and none is signed or sent, as in the network of posts of [9]. This is a rule of the design, and no result here proves it. The fibration does not prevent a global score by itself: in a star whose hub shares with every other unit, the hub is terminal in **Obs** and its fiber reindexes into every other. The rule is what keeps the hub's ranking its own.

Federation glues records, and each observer's trust stays its own.

Proposition 36 (Gluing of record sets)**.** *Let units $u_1, \ldots, u_n$ hold logs $L_1, \ldots, L_n$, and let A_i be the set of records whose audience includes u_i. (i) If $L_i \subseteq A_i$ and $L_i \cap A_j = L_j \cap A_i$ for all i, j, then $L = \bigcup_i L_i$ has $L \cap A_i = L_i$ for every i, and any view that is a function of a log agrees, on each L_i, with the view computed from $L \cap A_i$. (ii) Any set of records inside $\bigcup_i A_i$ whose intersection with every A_i is L_i equals $\bigcup_i L_i$. (iii) The logs lie inside their audiences and agree on overlaps exactly when there is a set L' of records inside $\bigcup_i A_i$ with $L_i = L' \cap A_i$ for every i.*

Phase	Input	Output	Next phase
none	invoice from clinic	post to family circle	drafting
drafting	she signs	signed request to clinic	awaiting clinic
drafting	family declines	nothing	declined
awaiting clinic	clinic co-signature	request, co-signature and pairing to bridge	sent
sent	settlement	receipt to clinic and family	settled

Table 2: The patient's box on one payment request. Its state is her log, and the phase is a view of the records about the request. Each row is one step of δ and λ.

Proof sketch. (i) $L \cap A_i = \bigcup_j (L_j \cap A_i) = \bigcup_j (L_i \cap A_j) \subseteq L_i$, and $L_i \subseteq L \cap A_i$ since $L_i \subseteq A_i$. The views agree because their arguments are equal. (ii) If $L' \subseteq \bigcup_i A_i$ restricts to every L_i, then $L' = \bigcup_i (L' \cap A_i) = \bigcup_i L_i$; this uses neither agreement on overlaps nor $L_i \subseteq A_i$. (iii) One direction is (i) with $L' = L$. For the other, $(L' \cap A_i) \cap A_j = (L' \cap A_j) \cap A_i$. □

Part (iii) is the sheaf condition for the presheaf of record sets on the cover $\{A_i\}$. Since identifiers are hashes, two logs can disagree on a shared record only by one of them lacking it, so the content of the condition is in the audience restriction.

Case study

Lab results. The lab's box emits a signed result to the patient's mailbox (Figure 1). Her box decides whether and when to pass it to her clinic; the clinic's box may reply to her. There is no wire from the lab to the clinic, so the wiring itself enforces her consent: a clinic reading a result holds an inhabitant of $\mathsf{Consented}(p, x, c)$ or nothing.

A payment request. When family abroad help pay a clinic bill, the clinic never sends them a request directly. The patient starts it: her box posts to her family circle, she signs a request, the clinic co-signs it, and her box forwards the request, the clinic's co-signature and the pairing record that names the clinic to her family, through a payment bridge, a box with one face toward the network and one toward a payment rail. Her box's state is her log, and Table 2 lists its steps for one request. The phase of the request is a view of the records about it, so it only moves forward. A family member accepts the request when it inhabits $\mathsf{PayRequest}(p, c, \mathit{item})$ in their own log, with c the clinic she is paired with (Example 19). The co-signature names the request by identifier, so order on the wire cannot make it count early.

AI agents in the model

Agent frameworks commonly put a supervisor in the middle that routes messages and assigns tasks. In this model each participant, human or AI, is a box; a team is a wiring diagram; and the plan is the pattern of wires together with the terms signed before work starts. An agent's box is nondeterministic inside, since a language model samples. What crosses a wire is a signed record, so once an agent's draft is recorded it can be checked like any other record. For the monotone parts of a collaboration, replaying the log reaches the same state in any order; a box whose state depends on order, like the one in Table 2, needs the delivery order as well. Treating language-model work as state machines has precedent [42]. Automata learning infers deterministic automata from queries [3], and Mealy machines in particular [33]. Since an agent

built on a language model is not deterministic, what can be learned this way is the protocol around it.

Related work

Machines, wiring and actors. Rutten's universal coalgebra treats state-based systems and bisimulation uniformly [30], and Mealy machines have a coalgebraic account of their specification and synthesis [6, 31]. Katis, Sabadini and Walters compose transition systems in series and in parallel [21], and Mealy machines with delayed feedback [22]. Our congruence result is stated for that composition. Actors [1, 15] and Erlang [4, 39] give networks that change shape as they run, and an Erlang state machine is a Mealy machine [11]. Our actors exchange signed records kept in an append-only log, and their wiring changes only through co-signed records.

Categorical systems theory. Wiring diagrams form an operad [36]. Open dynamical systems are algebras on operads of wiring diagrams, in continuous time with wires that pass their contents on at once [40], and in discrete time with wires that carry state [29]. Mode-dependent networks [38] and polynomial functors [28, 37] treat wiring that depends on the state of its parts, and Myers composes systems through lenses and wiring diagrams [27]. Shapiro and Spivak's dynamic organizations make the wiring a system of its own, with deep learning and prediction markets as examples [34]. We state a protocol that runs in these terms, with routing that reads pairings from positions and a registry of pairings as a coalgebra in O**rg**, and check both in Lean.

Replication without coordination. State that grows by union is a conflict-free replicated data type [35], local-first software keeps the primary copy on each user's device [24], and such types can be made to tolerate Byzantine peers [23]. The CALM theorem says that a problem has a consistent implementation without coordination exactly when it is monotone [14]. Ameloot, Neven and Van den Bussche prove it for networks of relational transducers [2], and Bloom^L carries monotone programming from sets to lattices [8]. Our form of the result covers Mealy boxes over signed records, outside inputs, wiring that grows through pairing ceremonies, and unboundedly many units, where each unit settles but the network may never fall quiet. It is proved in Lean.

Claims, types and provenance. Propositions as types [16, 41] reads a proof as a program and a proposition as its type. Provenance semirings annotate derived facts with how they were derived [12], and quantitative type theory grades a judgement's variables by a semiring [5]. Here a claim type is inhabited by signed records, its authority rule can depend on pairing records, and a grade is a set of signers in the semiring of positive Boolean expressions, which cannot count.

Agents and authority. Role-based access control [32] and the separation of duty in the Clark–Wilson model [7] decide who may do what. Statecharts [13] and state machines for language-model workflows [42] structure what a program does next, and automata learning infers a machine from its behaviour [3, 33]. In our model an agent is a box like any other party. Its key counts only once a person signs for it, and no agent can satisfy a reserved claim, so no supervisor sits in the middle.

Trust. EigenTrust computes one global trust value for each peer [18]. We index trust by its observer instead: the fibers of a split fibration [17] hold each observer's trust, gluing joins record sets along audiences, and a design rule keeps every ranking inside the fiber of the observer who computes it, as in the network of posts of [9].

What we do not claim

- New mathematics. Each result applies a known one to this model. What we add is a model of a system as it is built and a check of its results in Lean.
- That a language model inside a box is deterministic. Only its signed outputs are replayed.
- That agreement of two implementations on shared test vectors proves bisimilarity. It checks a finite part of the relation.
- That steps which are not monotone, revocation among them, converge without coordination. Proposition 28 says they do not, and the model asks for a signature there.
- That an observer's acceptance is anyone else's. Two observers trusting different keys can disagree about the same record, by design.

Conclusion

In this model a collaboration is a composition of the parties' own machines, and the results say what that composition gives. A party can be replaced by a bisimilar one without changing the collaboration (Theorem 24). A collaboration is a unit of a larger one, and its wiring, even as it changes, is one map of polynomials and a coalgebra in $\mathbb{O}\mathbf{rg}$ (Theorem 13 and Proposition 15). Monotone work reaches the same logs under any fair order with no coordinator (Theorem 26), and the steps that are not monotone are where the model asks for a signature (Proposition 28). Who may approve what is a question of typing (Propositions 20 and 21), and each observer's acceptance and trust stay its own (Theorem 31 and Proposition 35).

Some questions are open. A registry that routes only along pairings that still stand should make its presentation of the wiring agree with the one read from positions. The multicategory of Theorem 13 and the dynamic multicategory of Proposition 15 can be built as structures in Lean. Gluing should extend from one cover to a site whose covers come from pairings. Revocation removes a wire, so by Proposition 28 it needs coordination, and key epochs are one way to give it a monotone form.

Artifact

The propositions, lemmas and theorems of *The changing wiring in Poly* to *Trust as a fibration over observers* are mechanized in Lean 4, for the readings stated below. The remarks, the case study and the rule on rankings in *Trust as a fibration over observers* are not. One library uses core Lean only:

- `CosysCore/Basic`: polynomials, their maps, the tensor, dynamical systems and partitions;
- `CosysCore/Network`: Proposition 11 and Theorem 13;
- `CosysCore/Org`: Proposition 15;
- `CosysCore/Gluing`: Proposition 36;
- `CosysCore/Trust`: Proposition 35.

The other is on Mathlib:

- `CosysCalco/Mealy`: Theorem 24;
- `CosysCalco/Order`: Propositions 9 and 27 to 29 and Theorem 26;
- `CosysCalco/Claims`: Propositions 20 and 21 and Lemma 23;
- `CosysCalco/Grades`: Theorem 31.

For Theorem 13 (ii) and Proposition 15 the mechanization gives the identity and composition maps and proves their equations; it does not assemble them into a multicategory in Lean. Congruence is proved for boxes as Mealy machines over a stream of events, which covers every schedule at once. The order results are proved for monotone boxes at the level of their logs: a unit's state is its log and what it has emitted, which Definition 25 makes a function of the log, and the receive rule is the step. A lemma shows that such a box reads a set of records to the same state in any order. The decision procedure of Proposition 20 (i) is a plain function, and the artifact runs it on an example. Sets are not asked to be finite where finiteness plays no part. The proofs, and the libraries that implement the model, are published at `https://github.com/cosys-work/cosys`. This paper describes release `proofs-v1.0.0`, archived on Zenodo under the DOI 10.5281/zenodo.24026063 [20].

Use of AI tools

We used Claude, an AI model made by Anthropic, throughout this work. It drafted the Related work section and revised the text. It wrote much of the rewritten Lean, including the new receive rule in `Order.lean` and the decision procedure for claims, and repaired the Lean proofs. It also reviewed drafts and checked the references against their sources. The model is that of a system we built, and we decided what to define, claim and prove. The Lean kernel checks every mechanized result, and `Check.lean` prints the axioms each one rests on. We read and checked every statement and reference in the paper, and we take responsibility for its content.

References

[1] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986. doi: 10.7551/mitpress/1086.001.0001.

[2] Tom J. Ameloot, Frank Neven, and Jan Van den Bussche. Relational transducers for declarative networking. *Journal of the ACM*, 60(2):15:1–15:38, 2013. doi: 10.1145/2450142.2450151.

[3] Dana Angluin. Learning regular sets from queries and counterexamples. *Information and Computation*, 75(2):87–106, 1987. doi: 10.1016/0890-5401(87)90052-6.

[4] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, Sweden, 2003. TRITA-IMIT-LECS AVH 03:09.

[5] Robert Atkey. Syntax and semantics of quantitative type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018)*, pages 56–65. ACM, 2018. doi: 10.1145/3209108.3209189.

[6] Marcello M. Bonsangue, Jan J. M. M. Rutten, and Alexandra Silva. Coalgebraic logic and synthesis of Mealy machines. In Roberto Amadio, editor, *Foundations of Software Science*

and Computational Structures (FoSSaCS 2008), volume 4962 of *Lecture Notes in Computer Science*, pages 231–245. Springer, 2008. doi: 10.1007/978-3-540-78499-9_17.

[7] David D. Clark and David R. Wilson. A comparison of commercial and military computer security policies. In *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, pages 184–194. IEEE, 1987. doi: 10.1109/SP.1987.10001.

[8] Neil Conway, William R. Marczak, Peter Alvaro, Joseph M. Hellerstein, and David Maier. Logic and lattices for distributed programming. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC 2012)*, pages 1–14. ACM, 2012. doi: 10.1145/2391229.2391230.

[9] CoSys Work Pvt. Ltd. SOMA: Signed posts as hyperedges. Category, ontology and convergence in a network of public posts. `https://cosys.work/soma-whitepaper.pdf`, 2026. White paper, CC BY 4.0.

[10] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, second edition, 2002. doi: 10.1017/CBO9780511809088.

[11] Erlang/OTP. gen_statem behaviour. `https://www.erlang.org/doc/system/statem.html`, 2026. Erlang/OTP 29.1.2 documentation. Accessed November 2026.

[12] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2007)*, pages 31–40. ACM, 2007. doi: 10.1145/1265530.1265535.

[13] David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987. doi: 10.1016/0167-6423(87)90035-9.

[14] Joseph M. Hellerstein and Peter Alvaro. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9):72–81, 2020. doi: 10.1145/3369736.

[15] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In Nils J. Nilsson, editor, *Proceedings of the Third International Joint Conference on Artificial Intelligence (IJCAI 1973)*, pages 235–245, 1973. URL `https://www.ijcai.org/Proceedings/73/Papers/027B.pdf`.

[16] William A. Howard. The formulae-as-types notion of construction. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, 1980.

[17] Bart Jacobs. *Categorical Logic and Type Theory*, volume 141 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, 1999.

[18] Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina. The EigenTrust algorithm for reputation management in P2P networks. In *Proceedings of the Twelfth International Conference on World Wide Web (WWW 2003)*, pages 640–651. ACM, 2003. doi: 10.1145/775152.775242.

[19] Risav Karna. Giants and atlases, 2014. Unpublished manuscript, August 2014: the twelve-row matrix behind Co-Planetary Systems.

[20] Risav Karna. cosys: packages and Lean 4 proofs for Collaboration Is Composition. Zenodo, release proofs-v1.0.0, 2026. URL `https://doi.org/10.5281/zenodo.24026063`.

[21] Piergiulio Katis, Nicoletta Sabadini, and Robert F. C. Walters. Span(Graph): A categorical algebra of transition systems. In Michael Johnson, editor, *Algebraic Methodology and Software Technology (AMAST 1997)*, volume 1349 of *Lecture Notes in Computer Science*, pages 307–321. Springer, 1997. doi: 10.1007/BFb0000479.

[22] Piergiulio Katis, Nicoletta Sabadini, and Robert F. C. Walters. Feedback, trace and fixed-point semantics. *RAIRO – Theoretical Informatics and Applications*, 36(2):181–194, 2002. doi: 10.1051/ita:2002009.

[23] Martin Kleppmann. Making CRDTs Byzantine fault tolerant. In *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC 2022)*, pages 8–15. ACM, 2022. doi: 10.1145/3517209.3524042.

[24] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*, pages 154–178. ACM, 2019. doi: 10.1145/3359591.3359737.

[25] George H. Mealy. A method for synthesizing sequential circuits. *The Bell System Technical Journal*, 34(5):1045–1079, 1955. doi: 10.1002/j.1538-7305.1955.tb03788.x.

[26] Edward F. Moore. Gedanken-experiments on sequential machines. In Claude E. Shannon and John McCarthy, editors, *Automata Studies*, number 34 in Annals of Mathematics Studies, pages 129–153. Princeton University Press, 1956. doi: 10.1515/9781400882618-006.

[27] David Jaz Myers. Categorical Systems Theory, 2023. URL `https://www.davidjaz.com/Papers/DynamicalBook.pdf`. Book draft, last updated 3 September 2023.

[28] Nelson Niu and David I. Spivak. *Polynomial Functors: A Mathematical Theory of Interaction*. Number 498 in London Mathematical Society Lecture Note Series. Cambridge University Press, 2025. doi: 10.1017/9781009576734.

[29] Dylan Rupel and David I. Spivak. The operad of temporal wiring diagrams: Formalizing a graphical language for discrete-time processes, 2013. arXiv:1307.6894.

[30] J. J. M. M. Rutten. Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000. doi: 10.1016/S0304-3975(00)00056-6.

[31] J. J. M. M. Rutten. Algebraic specification and coalgebraic synthesis of Mealy automata. *Electronic Notes in Theoretical Computer Science*, 160:305–319, 2006. doi: 10.1016/j.entcs.2006.05.030.

[32] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.

[33] Muzammil Shahbaz and Roland Groz. Inferring Mealy machines. In Ana Cavalcanti and Dennis R. Dams, editors, *FM 2009: Formal Methods, Second World Congress*, volume 5850 of *Lecture Notes in Computer Science*, pages 207–222. Springer, 2009. doi: 10.1007/978-3-642-05089-3_14.

[34] Brandon T. Shapiro and David I. Spivak. Dynamic operads, dynamic categories: From deep learning to prediction markets. In Jade Master and Martha Lewis, editors, *Proceedings of the Fifth International Conference on Applied Category Theory (ACT 2022)*, volume 380 of

Electronic Proceedings in Theoretical Computer Science, pages 183–202. Open Publishing Association, 2023. doi: 10.4204/EPTCS.380.11.

[35] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011. doi: 10.1007/978-3-642-24550-3_29.

[36] David I. Spivak. The operad of wiring diagrams: Formalizing a graphical language for databases, recursion, and plug-and-play circuits, 2013. arXiv:1305.0297.

[37] David I. Spivak. Poly: An abundant categorical setting for mode-dependent dynamics, 2020. arXiv:2005.01894.

[38] David I. Spivak and Joshua Tan. Nesting of dynamical systems and mode-dependent networks. *Journal of Complex Networks*, 5(3):389–408, 2017. doi: 10.1093/comnet/cnw022. arXiv:1502.07380.

[39] Hans Svensson and Lars-Åke Fredlund. A more accurate semantics for distributed Erlang. In *Proceedings of the 2007 ACM SIGPLAN Workshop on Erlang*, pages 43–54. ACM, 2007. doi: 10.1145/1292520.1292528.

[40] Dmitry Vagner, David I. Spivak, and Eugene Lerman. Algebras of open dynamical systems on the operad of wiring diagrams. *Theory and Applications of Categories*, 30(51):1793–1822, 2015. URL `http://www.tac.mta.ca/tac/volumes/30/51/30-51abs.html`.

[41] Philip Wadler. Propositions as types. *Communications of the ACM*, 58(12):75–84, 2015. doi: 10.1145/2699407.

[42] Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows. In *First Conference on Language Modeling (COLM 2024)*, 2024. URL `https://openreview.net/forum?id=3nTbuygoop`. arXiv:2403.11322.

© 2026 Risav Karna and CoSys Work Pvt. Ltd. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).